

Pull Request (PR) Review Checklist - Code Reviewers

Use this checklist to guide your review. Reviewers at all levels are **not expected to catch everything**—the goal is to improve clarity, correctness, maintainability, and shared understanding.

Reminder: Clear logic reviews and well-phrased questions are just as valuable as running code.

1. Understand the PR

Before diving into the code, ensure the intent is clear.

- ☐ Do I understand **what this PR is trying to do**?
- ☐ Is the PR description clear about the **goal and scope**?
- ☐ If something is unclear, did I **ask a question instead of assuming**?

It is always okay to say: > “I’m not sure I understand—can you explain...?”

2. Logic & Behavior

Focus on whether the code makes sense and behaves as intended.

- ☐ Does the code **do what the author says it does**?
- ☐ Are there any obvious edge cases or missing checks?
- ☐ Do variable names, function names, and comments help explain the logic?

You do **not** need to run the code to provide a valuable logic review.

3. Be Specific in Comments

Effective reviews are easy to act on.

- ☐ Did I reference **specific lines, files, or functions**?
- ☐ Does each comment focus on **one issue only**?
- ☐ Did I clearly label comments as a **question, suggestion, or bug**?

Example:

> “Line 87: What happens here if `config` is missing this key?”

4. Communication Style

How feedback is delivered matters.

- ☐ Are my comments **clear and concise**?
- ☐ Did I avoid repeating the same point in multiple ways?
- ☐ Is my tone **respectful and collaborative**?

Assume positive intent—code reviews are about improving code, not judging people.

5. Actionability

Make it easy for the author to respond.

- ☐ Can the author clearly tell **what to do next**?
- ☐ If I suggested a change, did I explain **why it matters**?
- ☐ Did I distinguish **required changes** from **optional suggestions**?

Example:

> “Required for merge” vs. “Optional improvement”

6. When You’re Unsure

Uncertainty is normal—handle it explicitly.

- ☐ Did I ask a question instead of making assumptions?
- ☐ Did I phrase uncertainty as a **question**, not a statement?

Example:

> “I might be missing something—can you explain how this handles X?”

7. Final Check

Before submitting the review:

- ☐ Did I acknowledge something that was done well?
 - ☐ Did I keep the review **focused** and not over-engineered?
-

Key Takeaway

A good code review is **clear, specific, and kind**.

Reviewers at all levels help improve the codebase *and* grow shared understanding through thoughtful feedback.

Authors Antonia Chroni, PhD (@AntoniaChroni)

These tools and pipelines have been developed by the Bioinformatic core team at the St. Jude Children's Research Hospital. These are open access materials distributed under the terms of the BSD 2-Clause License, which permits unrestricted use, distribution, and reproduction in any medium, provided the original author and source are credited.